

CURSO 2023-24

Práctica 5: Grafos

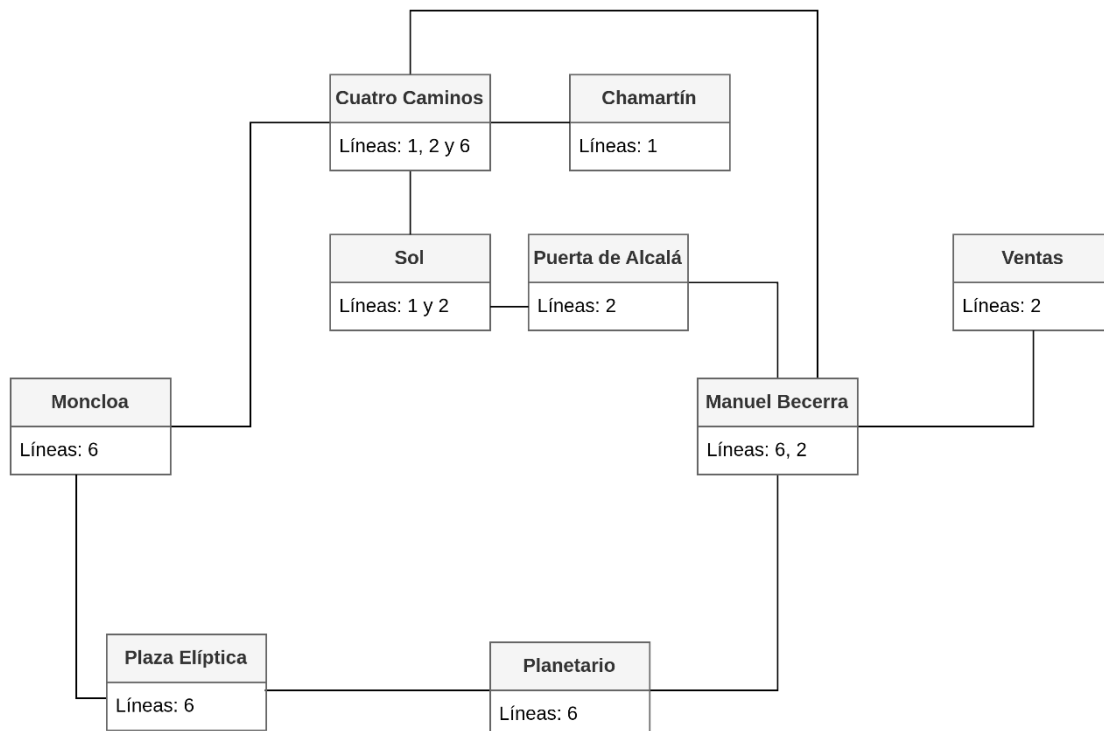
1. Objetivos

- Adquirir destreza en el manejo y su representación en forma de matriz de adyacencias.
- Uso de la clase GrafoMA (grafo implementado con una Matriz de adyacencia).
 - Para representar un conjunto de datos
 - Para aplicar algoritmos aprendidos en teoría a la resolución de situaciones

2. Definición y manejo de un grafo

En marzo va a asistir un grupo de alumnos de bachillerato de fuera a una visita a algunas zonas turísticas de Madrid. Para planificar el viaje, se han descargado el plano turístico del metro de Madrid ([Planoturistico.pdf \(metromadrid.es\)](#)) y están analizando el mapa para entender cómo es la estructura.

Finalmente han decidido que quieren visitar la plaza mayor, Sol, Puerta de Alcalá, Planetario y Ventas. Algunos llegan por tren a Chamartín y otros en autobús a plaza elíptica. Así, han simplificado el metro a las paradas que les interesa como sigue:



Ejercicio 1: Creación de las clases necesarias para el manejo de los datos

Para poder analizar el mapa, incluye las paradas y las líneas de metro correspondientes, representando el grafo siguiente. Incluye en cada vértice las líneas que pasan por el mismo.

Para ello, sigue los pasos siguientes:

1. Descarga el proyecto **ED-5-PracticaGrafos** y ábrelo con IntelliJ IDEA.
2. El proyecto contiene una carpeta principal “metro” con:
 - a. Cola.java: el mismo fichero que se usa en las clases de teoría para grafos.
 - b. GrafoMA.java: el mismo fichero que se usa en las clases de teoría.
 - c. Metro.java: servirá para alojar la información asociada a una red de metro.
 - d. Nodo.java: el mismo fichero que se usa en las clases de teoría para grafos.
 - e. Parada.java: servirá para alojar la información de una parada de metro.
 - f. Pruebas.java.
3. La clase Pruebas incluye la función *main* para introducir dentro las diferentes ejecuciones que se solicitarán a lo largo de la práctica. Para facilitar la lectura, separa cada ejercicio en un método *public static void ejercicio_<>()* diferente y realiza llamadas desde *main* a las funciones definidas.
4. Utilizando la clase *GrafoMA*, completa las clases siguientes con los atributos y métodos de construcción y obtención de información propuestos.

Clase	Atributos	Métodos
Parada	<i>private String nombre</i> Indica el nombre de la parada <i>private int [] líneas</i> Indica las líneas que pasan por la parada <i>private int numLineas</i> Indica el número de líneas que pasan por la parada	<i>public Parada (String nombre, int[] lineas)</i> setNombre, setLineas (incluirá actualizar el número de líneas) getNombre, getLineas, getNumLineas. <i>@Override</i> <i>public String toString(){}</i> Debe retornar una cadena con la información de la parada. Esto es: Parada{ <nombre> , Líneas: <>, num_lineas}
Metro	<i>private GrafoMA redMetro</i> Contiene el grafo que representa el esquema de las paradas <i>private Parada[] paradas</i> Contiene la información de cada parada <i>private int numParadas</i>	<i>public Metro(Parada[])</i> Inicializará <i>redMetro</i> y <i>paradas</i> en función de la lista de paradas que se le pase. <i>public int indiceParada(Parada parada)</i> Busca la posición en la lista de vértices de la parada introducida. Si no existe, devuelve -1 <i>public comunicarParadas(Parada p1, Parada p2)</i> Usando <i>indiceParada</i>, añade la arista <i>p1 -> p2</i> a <i>redMetro</i> <i>public int getNumParadas()</i> <i>public void mostrarAmpliado()</i> que haga <i>redMetro.mostrarAmpliado()</i>

5. Para comprobar que se ha creado correctamente ambas clases:
 - a. Crea las paradas planetario y plaza elíptica e imprime su contenido utilizando el método *toString()* que has definido.



- Crea el grafo *planetario* <-> *plaza elíptica* usando el constructor de **Metro** así como el método **Metro.comunicarParadas()**
- Imprime la matriz de adyacencia usando **Metro.mostrarAmpliado()**.
- Antes de continuar. Asegúrate de comprender lo que significa la matriz de adyacencia que has obtenido al utilizar **mostrarAmpliado()**: ¿Qué es lo que aparece? ¿Qué significan los números? ¿Y las letras? ¿En qué se nota que es un grafo no dirigido?

En la clase Prueba, el resultado de la ejecución con

```
public static void main(String[] args) {  
    System.out.println("Práctica 5 grafos");  
    ejercicio_1();  
    //ejercicio_2();  
    //ejercicio_3();  
}
```

debe ser el siguiente:

```
----- Ejercicio 1 -----  
-- Crear Paradas --  
Parada{Planetario Líneas: [6], 1}  
Parada{Plaza Elíptica Líneas: [6], 1}  
-- Crear red --  
-- Comunicar Paradas --  
-- Información de la red de metro --  
El grafo tiene una Matriz de 2 x 2  
De un grafo No dirigido  
  0 1  
0 F T  
1 T F  
  
Process finished with exit code 0
```

Ejercicio 2. Representación de los datos utilizando las clases propuestas y manejo de grafos no dirigidos

- Para el desarrollo de este apartado define el método `public static void ejercicio2()` en la clase Pruebas e incluye dentro todas las pruebas.
- Añade todas las paradas del grafo propuesto usando los métodos de **Metro**.
- Antes de continuar, verifica la matriz de adyacencia y el número de vértices (9) imprimiendo ambos por pantalla. Si ejecutas todo el ejercicio 2 hasta aquí debes obtener:

```
----- Ejercicio 2 -----  
-- Crear Paradas --  
-- Listar Paradas --  
Paradas (Usando Arrays.toString):  
[Parada{Planetario Líneas: [6], 1}, Parada{Plaza Elíptica Líneas: [6], 1},  
Parada{Moncloa Líneas: [6], 1}, Parada{Manuel Becerra Líneas: [6, 2], 2}, Parada{Sol  
Líneas: [1, 2], 2}, Parada{Alcala Líneas: [2], 1}, Parada{Ventas Líneas: [2], 1},  
Parada{Cuatro Caminos Líneas: [1, 2, 6], 3}, Parada{Chamartin Líneas: [1], 1}]  
-- Crear redMetro --
```

-- Comunicar Paradas --

Numero de paradas: 9

El grafo tiene una Matriz de 9 x 9

De un grafo No dirigido

	0	1	2	3	4	5	6	7	8
0	F	T	F	T	F	F	F	F	F
1	T	F	T	F	F	F	F	F	F
2	F	T	F	F	F	F	F	T	F
3	T	F	F	F	F	T	T	T	F
4	F	F	F	F	F	T	T	F	F
5	F	F	F	T	T	F	F	F	F
6	F	F	F	T	F	F	F	F	F
7	F	F	T	T	T	F	F	F	T
8	F	F	F	F	F	F	F	T	F

4. Define en Metro un método `public LinkedList <Parada> conexiones(Parada parada)` que dada una parada devuelva un `LinkedList` con todas las paradas que conecta. Comprueba que funciona imprimiendo en **Pruebas** su resultado para el caso de la parada *planetario*. La ejecución **miRed.conexiones(planetario)** (donde *planetario* es el nombre que le hayas dado a la variable que contiene la parada de Planetario) debe mostrar:

[Parada{Plaza Elíptica Líneas: [6], 1}, Parada{Manuel Becerra Líneas: [6, 2], 2}]

5. Define un método `public LinkedList<LinkedList <Parada>> getConexiones()` que devuelva las conexiones de cada vértice del grafo (**miRed.conexiones(paradas[i])** para cada vértice i).

Asimismo, define un método `public void mostrarConexiones()` que se apoye en el anterior para mostrar por pantalla el array de conexiones. Al ejecutarlo en pruebas, deberás obtener el resultado siguiente (salvo posible cambio de orden al introducir las paradas. Intenta que salga igual.):

```
--> Todas las conexiones de mi red de metro <--
0: Planetario conecta con: Plaza Elíptica Manuel Becerra
1: Plaza Elíptica conecta con: Planetario Moncloa
2: Moncloa conecta con: Plaza Elíptica Cuatro Caminos
3: Manuel Becerra conecta con: Planetario Alcala Ventas Cuatro Caminos
4: Sol conecta con: Alcala Cuatro Caminos
5: Alcala conecta con: Manuel Becerra Sol
6: Ventas conecta con: Manuel Becerra
7: Cuatro Caminos conecta con: Moncloa Manuel Becerra Sol Chamartin
8: Chamartin conecta con: Cuatro Caminos
```

6. Define `mostrarConexionesLineas()` que haga lo mismo que `mostrarConexiones()` añadiendo detrás de cada plaza entre paréntesis la línea por la que están unidas las dos paradas de metro. Puedes definir una función auxiliar `private LinkedList<Integer> lineasComunes(Parada parada1, Parada parada2)` que devuelva las líneas que tienen en común dos paradas. El resultado final debería ser el siguiente:

```
--> Todas las conexiones de mi red de metro (incluye líneas) <--
0: Planetario conecta con: Plaza Elíptica ([6]) Manuel Becerra ([6])
1: Plaza Elíptica conecta con: Planetario ([6]) Moncloa ([6])
2: Moncloa conecta con: Plaza Elíptica ([6]) Cuatro Caminos ([6])
3: Manuel Becerra conecta con: Planetario ([6]) Alcala ([2]) Ventas ([2]) Cuatro Caminos ([6, 2])
```



4: Sol conecta con: Alcala ([2]) Cuatro Caminos ([1, 2])
5: Alcala conecta con: Manuel Becerra ([2]) Sol ([2])
6: Ventas conecta con: Manuel Becerra ([2])
7: Cuatro Caminos conecta con: Moncloa ([6]) Manuel Becerra ([2, 6]) Sol ([1, 2]) Chamartin ([1])
8: Chamartin conecta con: Cuatro Caminos ([1])

7. Razona: ¿qué significa, en este caso, el grado de un vértice? ¿qué grado tendría cada parada?
8. Para cada vértice del grafo, encuentra su grado. Para ello, define en **Metro** un método público `int[] getGrados()` usando un método disponible en **GrafoMA**. Al ejecutarlo debes obtener el siguiente mensaje (usando `println` dentro del propio método `getGrados`, en el bucle que recorre los vértices para obtener el grado de cada uno):

Grado Planetario: 2
Grado Plaza Elíptica: 2
Grado Moncloa: 2
Grado Manuel Becerra: 4
Grado Sol: 2
Grado Alcala: 2
Grado Ventas: 1
Grado Cuatro Caminos: 4
Grado Chamartin: 1

Devolviendo un array con: {2,2,2,4,2,2,1,4,1}.

Ejercicio 3. Manejo de grafos dirigidos

1. Define `public static void ejercicio3()` para incluir dentro las ejecuciones que se piden a continuación.
2. Supongamos que se ha estropeado el metro entre las paradas Manuel Becerra y Cuatro Caminos en sentido a Cuatro Caminos.
3. Modifica el grafo para que muestre la información siguiendo los pasos siguientes:
 - a. Define un método `private void hacerDirigido()` en la clase **Metro** que sustituya la red `redMetro` por una idéntica pero dirigida. Si ya es dirigida no es necesario modificarla.
 - b. Define un método dentro de **Metro** `obrasEntreParadas(Parada origen Parada destino)` que convierta el grafo en dirigido y elimine la conexión entre origen y destino.
 - c. Ejecuta el método y comprueba la salida utilizando el caso propuesto (evitar Manuel becerra -> cuatro Caminos). El resultado debe ser

El grafo tiene una Matriz de 9 x 9

De un grafo dirigido

	0	1	2	3	4	5	6	7	8
0	F	T	F	T	F	F	F	F	F
1	T	F	T	F	F	F	F	F	F



```
2 F T F F F F F T F
3 T F F F F T T F F
4 F F F F F T F T F
5 F F F T T F F F F
6 F F F T F F F F F
7 F F T T T F F F T
8 F F F F F F F T F
```

4. Para poderse encontrar, los dos grupos de amigos deciden verse en Moncloa. Haz un método *public boolean existeCamino(Parada p1, Parada p2)* que compruebe, usando recorrido en profundidad, si se puede ir desde p1 hasta p2.

Comprueba, mediante su ejecución, si se puede ir desde Chamartin y Plaza Elíptica (lugares de llegada de cada grupo) hasta Moncloa (lugar de encuentro).

Desde Pruebas (con los mensajes de impresión en pruebas), se verá así:

```
¿Se puede ir de Chamartin a Moncloa? true
¿Se puede ir de Plaza Elíptica a Moncloa? true
```

5. Supongamos que, además de las anteriores, hay obras desde Plaza Elíptica hacia Moncloa y desde Manuel Becerra hacia Puerta de Alcalá. ¿Podrán encontrarse los dos grupos o necesitarán verse en algún lugar alternativo?

Ejecuta el código necesario para responder a esta pregunta a partir de los métodos definidos anteriormente en este apartado.

La ejecución completa del ejercicio 3 desde Pruebas debe mostrar lo siguiente:

```
----- Ejercicio 3 -----
-- Crear Paradas --
-- Listar Paradas --
Paradas (Usando Arrays.toString):
[Parada{Planetario Líneas: [6], 1}, Parada{Plaza Elíptica Líneas: [6], 1},
Parada{Moncloa Líneas: [6], 1}, Parada{Manuel Becerra Líneas: [6, 2], 2}, Parada{Sol
Líneas: [1, 2], 2}, Parada{Alcala Líneas: [2], 1}, Parada{Ventas Líneas: [2], 1},
Parada{Cuatro Caminos Líneas: [1, 2, 6], 3}, Parada{Chamartin Líneas: [1], 1}]
-- Crear redMetro --
-- Comunicar Paradas --
Numero de paradas: 9
El grafo tiene una Matriz de 9 x 9
De un grafo No dirigido
0 1 2 3 4 5 6 7 8
```



0 F T F T F F F F F

1 T F T F F F F F F

2 F T F F F F F T F

3 T F F F F T T T F

4 F F F F F T F T F

5 F F F T T F F F F

6 F F F T F F F F F

7 F F T T T F F F T

8 F F F F F F F T F

--> Obras desde Manuel Becerra hacia Cuatro Caminos <--

El grafo tiene una Matriz de 9 x 9

De un grafo dirigido

0 1 2 3 4 5 6 7 8

0 F T F T F F F F F

1 T F T F F F F F F

2 F T F F F F F T F

3 T F F F F T T F F

4 F F F F F T F T F

5 F F F T T F F F F

6 F F F T F F F F F

7 F F T T T F F F T

8 F F F F F F F T F

¿Se puede ir de Chamartin a Moncloa? true

¿Se puede ir de Plaza Elíptica a Moncloa? true

--> Nuevas obras

El grafo tiene una Matriz de 9 x 9

De un grafo dirigido

0 1 2 3 4 5 6 7 8

0 F T F T F F F F F

1 T F F F F F F F F

2 F T F F F F F T F

3 T F F F F F T F F



4 F F F F F T F T F

5 F F F T T F F F F

6 F F F T F F F F F

7 F F T T T F F F T

8 F F F F F F F T F

¿Se puede ir de Chamartin a Moncloa? true

¿Se puede ir de Plaza Elíptica a Moncloa? false

¿Se pueden ver en Moncloa? false

Process finished with exit code 0

3. Revisión final del output de las pruebas propuestas

Finalmente, si ejecutas el código de la práctica completo, obtendrás:

----- Ejercicio 1 -----

-- Crear Paradas --

Parada{Planetario Líneas: [6], 1}

Parada{Plaza Elíptica Líneas: [6], 1}

-- Crear red --

-- Comunicar Paradas --

-- Información de la red de metro --

El grafo tiene una Matriz de 2 x 2

De un grafo No dirigido

0 1

0 F T

1 T F

----- Ejercicio 2 -----

-- Crear Paradas --

-- Listar Paradas --

Paradas (Usando Arrays.toString):

[Parada{Planetario Líneas: [6], 1}, Parada{Plaza Elíptica Líneas: [6], 1}, Parada{Moncloa Líneas: [6], 1}, Parada{Manuel Becerra Líneas: [6, 2], 2}, Parada{Sol Líneas: [1, 2], 2}, Parada{Alcala Líneas: [2], 1}, Parada{Ventas Líneas: [2], 1}, Parada{Cuatro Caminos Líneas: [1, 2, 6], 3}, Parada{Chamartin Líneas: [1], 1}]

-- Crear redMetro --



-- Comunicar Paradas --

Numero de paradas: 9

El grafo tiene una Matriz de 9 x 9

De un grafo No dirigido

	0	1	2	3	4	5	6	7	8
0	F	T	F	T	F	F	F	F	F
1	T	F	T	F	F	F	F	F	F
2	F	T	F	F	F	F	F	T	F
3	T	F	F	F	F	T	T	T	F
4	F	F	F	F	F	T	F	T	F
5	F	F	F	T	T	F	F	F	F
6	F	F	F	T	F	F	F	F	F
7	F	F	T	T	T	F	F	F	T
8	F	F	F	F	F	F	F	T	F

Conexiones con Plaza Elíptica

[Parada{Plaza Elíptica Líneas: [6], 1}, Parada{Manuel Becerra Líneas: [6, 2], 2}]

--> Todas las conexiones de mi red de metro <--

0: Planetario conecta con: Plaza Elíptica Manuel Becerra

1: Plaza Elíptica conecta con: Planetario Moncloa

2: Moncloa conecta con: Plaza Elíptica Cuatro Caminos

3: Manuel Becerra conecta con: Planetario Alcala Ventas Cuatro Caminos

4: Sol conecta con: Alcala Cuatro Caminos

5: Alcala conecta con: Manuel Becerra Sol

6: Ventas conecta con: Manuel Becerra

7: Cuatro Caminos conecta con: Moncloa Manuel Becerra Sol Chamartin

8: Chamartin conecta con: Cuatro Caminos

--> Todas las conexiones de mi red de metro (incluye líneas) <--

0: Planetario conecta con: Plaza Elíptica ([6]) Manuel Becerra ([6])

1: Plaza Elíptica conecta con: Planetario ([6]) Moncloa ([6])

2: Moncloa conecta con: Plaza Elíptica ([6]) Cuatro Caminos ([6])

3: Manuel Becerra conecta con: Planetario ([6]) Alcala ([2]) Ventas ([2]) Cuatro Caminos ([6, 2])

4: Sol conecta con: Alcala ([2]) Cuatro Caminos ([1, 2])



```
5: Alcala conecta con: Manuel Becerra ([2]) Sol ([2])
6: Ventas conecta con: Manuel Becerra ([2])
7: Cuatro Caminos conecta con: Moncloa ([6]) Manuel Becerra ([2, 6]) Sol ([1, 2])
Chamartin ([1])
8: Chamartin conecta con: Cuatro Caminos ([1])
--> Grados de los vértices <--

Grado Planetario: 2
Grado Plaza Elíptica: 2
Grado Moncloa: 2
Grado Manuel Becerra: 4
Grado Sol: 2
Grado Alcala: 2
Grado Ventas: 1
Grado Cuatro Caminos: 4
Grado Chamartin: 1

----- Ejercicio 3 -----

-- Crear Paradas --
-- Listar Paradas --

Paradas (Usando Arrays.toString):

[Parada{Planetario Líneas: [6], 1}, Parada{Plaza Elíptica Líneas: [6], 1}, Parada{Moncloa
Líneas: [6], 1}, Parada{Manuel Becerra Líneas: [6, 2], 2}, Parada{Sol Líneas: [1, 2], 2},
Parada{Alcala Líneas: [2], 1}, Parada{Ventas Líneas: [2], 1}, Parada{Cuatro Caminos
Líneas: [1, 2, 6], 3}, Parada{Chamartin Líneas: [1], 1}]

-- Crear redMetro --
-- Comunicar Paradas --

Numero de paradas: 9
El grafo tiene una Matriz de 9 x 9
De un grafo No dirigido

  0 1 2 3 4 5 6 7 8
0 F T F T F F F F F
1 T F T F F F F F F
2 F T F F F F F T F
3 T F F F F T T T F
```



4 F F F F F T F T F

5 F F F T T F F F F

6 F F F T F F F F F

7 F F T T T F F F T

8 F F F F F F F T F

--> Obras desde Manuel Becerra hacia Cuatro Caminos <--

El grafo tiene una Matriz de 9 x 9

De un grafo dirigido

0 1 2 3 4 5 6 7 8

0 F T F T F F F F F

1 T F T F F F F F F

2 F T F F F F F T F

3 T F F F F T T F F

4 F F F F F T F T F

5 F F F T T F F F F

6 F F F T F F F F F

7 F F T T T F F F T

8 F F F F F F F T F

¿Se puede ir de Chamartin a Moncloa? true

¿Se puede ir de Plaza Elíptica a Moncloa? true

--> Nuevas obras

El grafo tiene una Matriz de 9 x 9

De un grafo dirigido

0 1 2 3 4 5 6 7 8

0 F T F T F F F F F

1 T F F F F F F F F

2 F T F F F F F T F

3 T F F F F F T F F

4 F F F F F T F T F

5 F F F T T F F F F

6 F F F T F F F F F

7 F F T T T F F F T



8 F F F F F F F T F

¿Se puede ir de Chamartin a Moncloa? true

¿Se puede ir de Plaza Eliptica a Moncloa? false

¿Se pueden ver en Moncloa? false

Process finished with exit code 0

4. Entrega de la práctica

1. Se entregará solamente el proyecto IntelliJ resultante de hacer la práctica “ED-5-PracticaGrafos”, que ha de tener exactamente ese nombre. Tu nombre y el grupo al que perteneces deberá aparecer como comentario al principio de la clase Metro.
2. Para entregarlo, se comprimirá en un archivo, preferentemente zip, y se subirá a la plataforma. El nombre del archivo ZIP será exactamente el mismo: “ED-5-PracticaGrafos.zip”. Por favor, revísalo al momento de subirlo, Moodle permite modificar el nombre del .zip en ese paso.
3. No olvide que el proyecto debe incluir las pruebas solicitadas en este enunciado, incluidas en el código del método *main* en las Clase Prueba. Ten cuidado de no modificar las clases predefinidas y seguir correctamente las especificaciones (métodos, resultados, cabeceras de métodos, etc.)